

《借灯》

一盏灯，照见一段被掩埋的真相

项目文档

v1.0

课程名称 网站开发小学期

指导教师 赵丰年

项目组 白灯客

小组成员 罗晨菲、杨梦、于知让、高冰轩、卢正松

2026 年 9 月

目录

- 一、 项目概述 4
 - 1.1 游戏简介 4
 - 1.1.1 概述 4
 - 1.1.2 核心玩法循环 5
 - 1.1.3 创作背景与理念 5
 - 1.2 体验定位 6
 - 1.2.1 玩家画像 6
 - 1.2.2 沉浸式体验 6
 - 1.2.3 感受主旨 7
 - 1.3 项目目标分期与实现边界 7
 - 1.3.1 分期实现 7
 - 1.3.2 实现难点与挑战 8
- 二、 游戏设计详解 10
 - 2.1 主线剧情 10
 - 2.1.1 开场与物品 10
 - 2.1.2 三条身份线 10
 - 2.1.3 五次揭示顺序 10
 - 2.1.4 五结局设计 11
 - 2.2 探索场景 12
 - 2.2.1 场景即界面 12
 - 2.2.2 场景清单 12
 - 2.2.3 人物对话 13
 - 2.2.4 背包与线索 13
 - 2.3 核心交互 14
 - 2.3.1 剧情对话 14
 - 2.3.2 四种小游戏 14
 - 2.3.3 恐怖表现与设计 14
 - 2.3.4 存档与进度恢复 15
 - 2.4 附加功能 15
 - 2.4.1 终局追逐与生命值系统 15

2.4.2	成就系统	15
2.4.3	项目组信息	17
2.4.4	音效与美术	18
三、	美学与用户体验	19
3.1	视觉设计规范	19
3.1.1	整体风格	19
3.1.2	配色	19
3.1.3	字体	19
3.1.4	人物立绘	20
3.1.5	美术原则	21
3.1.6	动效规范	21
3.2	文字风格	22
3.2.1	整体调性	22
3.2.2	用语	22
3.2.3	氛围营造	22
3.3	界面设计	23
3.3.1	站点地图	23
3.3.2	线框图	23
3.3.3	设计图	23
3.4	恐怖美学设计	24
3.4.1	整体特点	24
3.4.2	音效	24
3.4.3	视频	25
3.4.4	转场	25
四、	技术方案与质量保证	27
4.1	开发与运行平台	27
4.2	技术选型	27
4.2.1	原生技术栈	27
4.2.2	浏览器本地存储	27
4.3	技术架构	28
4.4	代码组织	29
4.5	质量保证	30
4.5.1	异常与降级	30
4.5.2	测试	31

4.5.3 协作流程与 subPR 分工	31
五、 项目管理与进度	32
5.1 项目提出与开发方式	32
5.2 协作流程	32
5.2.1 subPR 的运作规则	32
5.2.2 五日迭代节奏	33
5.3 轮值角色与分工	33
5.4 迭代进度	35
5.5 流程效果	35
六、 测试与回帖情况总结	37
6.1 浏览器兼容测试	37
6.2 操作系统兼容测试	37
6.3 分辨率兼容测试	37
6.4 回帖情况汇总	38
七、 最终显示效果	41
八、 小组总结	42
九、 个人总结	47
十、 评优申请	48
十一、 最终得分	49

一、项目概述

1.1 游戏简介

1.1.1 概述

《借灯》是一款用原生 HTML5、CSS3 与 JavaScript 写成的浏览器端交互式悬疑叙事游戏。项目不依赖任何前端框架，也没有构建步骤，源码即产物。开发过程按企业级轻量化项目的规范组织：需求先落成文档、拆成 subPR 再动手，代码走功能分支与 PR 评审，五名成员在四个版本中轮换承担项目经理、产品经理、技术负责人等职责。

故事发生在虚构的皖赣交界废弃矿山古村石涧村。村里流传着一条禁忌：死在外地的人若找不到归途，会在雨夜来借一盏没有火焰的白灯笼；灯不能碰，门不能开，更不能回应门外叫自己名字的声音，否则死人会借走你的脸。玩家扮演一个在暴雨后从祠堂醒来的失忆者，身上只有一张烧毁大半、仅剩姓氏可辨的工作证和一颗蓝玻璃珠。他要在查清村庄闹鬼真相的同时，回答一个更麻烦的问题：我是谁。

项目想在三个方向上都有所交代：工程上有一套可追溯的规范流程，叙事上有一个经得起反复推敲的反转结构，主题上留下一点值得回味的感受。



图 1.1 石涧村设计图

1.1.2 核心玩法循环

游戏的基本循环是一条闭环，玩家的每一步操作最终都落在同一份进度记录上：

阅读剧情 → 探索场景 → 调查与对话 → 收集线索 → 完成小游戏 → 作出抉择

走完这一轮，剧情推进到下一个节点，循环重新开始。

阅读环节按段落推进剧情，对话带发言者名牌，独白与旁白分开渲染。探索环节里，场景中的发光热点是唯一的交互入口，点击物品或人物会触发调查、对话或线索详情，得到的物证进入背包。小游戏嵌在剧情节点上，地图复原、拆鬼、矿井通路和终局追逐都在此列，结果会回写到剧情状态。

1.1.3 创作背景与理念

故事来自小组的原创悬疑剧本，表层框架借用了南方山村雨夜借灯的民俗传说，氛围落在废弃矿山古村的落寞上，重心则始终放在人物的命运纠葛和选择上。

石涧村的这条禁忌是完整的：死者在外地找不到归途，会在雨夜借一盏没有火焰的白灯；活人若开门回应，脸就会被借走。它既是恐怖氛围的来源，也是叙事的机关。白灯笼、广播里的童谣、只有进没有出的湿脚印、墙上写着的已故村民名字，这些在前半段看起来无法解释的现象，故事进行到中段会告诉玩家它们全都有物理成因。真正推动悲剧的，是集团、村民、父亲和主角各自作出的选择。白灯象征的不是超自然力量，而是人借传说掩盖责任。

主人公的一生是一条闭合的因果链，每个阶段的决定都在为最后的结局埋线。

童年时矿难夺走了他的亲人，家庭随之破碎。他选择逃离故乡，毁容改名，把过去整个切开，以为这样就能重新活一次。创伤并没有因此愈合，只是被压到了意识深处。

成年后他被迫回到故地，故乡的一草一木把深埋的恐惧全部唤了回来。他没有选择直面，而是照着童年的记忆复刻出一套恐吓装置，把自己最怕的东西变成了威胁别人的工具。

为了掩盖罪行，他又分裂出白灯客这个匿名身份，抹掉作恶的证据，试图在道德上把自己和做过的事切开。这个身份既是自保的手段，也是人格崩塌的标志。

全程的沉默并没有换来救赎。他越是逃避，越是被迫面对；越是沉默，罪责越重。自我救赎的失败，构成了整条命运逻辑的闭环。



图 1.2 主角的矛盾人生设计

1.2 体验定位

1.2.1 玩家画像

《借灯》面向的是愿意花两三个小时读完一个完整故事的人。这一版预计游玩时长三小时左右，节奏由文本和调查推动，不设操作门槛：V2 之后核心操作只剩鼠标点击，键盘只用于少量快捷键。

游戏不要求玩家事先了解任何背景知识，也不依赖前作。三小时的流程里，前三分之一用来建立悬念，中间用来推翻玩家已经建立的判断，最后一段留给一个没有标准答案的决定。

1.2.2 沉浸式体验

沉浸感的建立方式，主要是在几个地方不给玩家退路。

开场失忆让玩家和主角站在完全相同的起点。你不知道自己是谁，村民对你的称呼也互相矛盾：多数人认定陈晋年十七年前就死了，只认识来谈拆迁的王工程师；一个记忆混乱的老人会突然喊出陈晋年的名字。玩家手里的信息、心里的疑问，和屏幕里的主角一模一样。

游戏里没有太多靠突然的画面和音效吓人。恐怖感来自另一件事：中段揭露所有灵异现象都是人为装置之后，玩家会认出这些装置的元素全部来自主角的童年记忆。我们希望做出这样“可怕的不是鬼，而是人”的艺术反转效果，从而引发玩家的思考与喜爱。

场景细节同样在为这件事服务。檐下的雨声、排水渠里流动的水、广播里带着电流杂音的童谣、雨后只有进没有出的湿脚印，每一样都同时属于村庄的日常和恐吓装置的一部分。

界面上则尽量不打断玩家。同一时刻只允许存在一种主状态，线索详情、背包或小游戏窗口打开时，背后的场景会被锁住，关闭后回到原来的状态。

1.2.3 感受主旨

故事表面上问的是村里有没有鬼，中段变成谁在装鬼，后段揭晓装鬼的人就是主角。真正的问题留到了最后：他为什么需要一个鬼。

只要存在一个外在的、说不清的力量在作祟，他就不用面对另一件事，换句话说，这个村子里所有的苦难，都是活人做出来的选择。

终局的处理方式呼应了这一点。玩家不是从几个结局里选一句话，而是决定保留哪几组证据、删掉哪几组。当玩家成功逃离小周之后，销毁全部、选择性公开、完整公开，三条路的分歧在于玩家愿意承担多少。完整公开意味着同时承认十七年前的矿难、集团的罪行、父亲的隐瞒，以及自己设计恐吓、拒绝施救和删改证据的全部事实。

我们希望玩家在通关之后，对沉默和逃避这两个词多一点警觉，这是这个项目最想留下的一点东西。

1.3 项目目标分期与实现边界

1.3.1 分期实现

项目分四个版本迭代，每一代有明确的交付范围和明确的不做清单。

V1 解决的是有没有的问题，从零到一。站点能跑，剧情能串联，核心是验证可行性。这一代以故事大纲中的 14 个探索阶段作为全局口径，V1 只完成前三个：祠堂醒来、村口开放、陈家老宅。同时交付登录、注册、游客三种进入方式，主菜单、成就、项目介绍、制作组以及五名成员各自的独立页面，共计八个以上 HTML 页面。小游戏做了手绘地图复原一个，成就系统做到能解锁第一项，存档先只存在浏览器本地。V1 明确不做的部分，向后的剧情，第二个小游戏、完整证据链、全部身份反转和多结局等等。

V2 不新增任何剧情，只重做游戏内的交互方式。界面被拆成五种互斥状态：看剧情、逛场景、看线索详情、看背包、玩小游戏。同一时刻只能处于其中一种，弹窗打开时背后的场景锁死。探索从列表式操作改成直接点击场景里的发光热点。这一版还统一了进度来源，storyCheckpoint 成为唯一的剧情进度记录，其他字段只作为由它生成的只读投影存在，避免全局、剧情和探索各记一份互相打架的进度。

V3 把故事推到结局。老宅不再是整局的终点，而是后半段调查的入口。这一代新增 28 个运行时 Node 和四个后续剧情阶段，把三条身份线的外围调查接入正

式流程，新做三个小游戏，并把五个结局全部跑通。它的技术前提是 V2 必须先验收并合并入 main，V3 分支从包含完整 V2 的最新 main 上重建，不允许在缺失 V2 的旧分支上批量开工。

V4 解决的是好不好问题，不再增加结构性内容，做的是内容改写与体验收尾。剧情对话按 46 个改写块全面修订，覆盖序幕到终局的全流程；对话框补上响应式；新增 11 处转场效果；插入过场视频；菜单页、成就页和背包页的展示细节陆续调整。这一版还处理了一个具体的渲染问题：探索热点被点击后可能走阅读视图或物品详情弹窗两条不同的路径，两条路径对多行文本和发言者名牌的处理规则并不相同，写文案时必须先判断这段话会走哪条路。

表 1.1 四个版本的目标与交付

版本	版本目标	交付效果
V1	搭起站点骨架，跑通前三阶段	8 个以上独立页面；登录、注册、游客三条入口；祠堂、村口、陈家老宅可连续游玩；地图复原小游戏；最小可用成就系统；本地存档
V2	重构游戏内交互，不新增剧情	五种互斥界面状态；点击发光热点探索；storyCheckpoint 成为唯一进度来源；1280px 与 390px 双档验收
V3	从 week-one-end 推进到正式结局	28 个运行时 Node；四个后续剧情阶段；四条外围调查线；三个新增小游戏；五个稳定结局
V4	剧情改写与体验打磨	46 处剧情对话改写全部落地；小游戏二期；对话框响应式；11 处转场效果；过场视频插入；菜单、成就与背包页细节调整，视听与美术效果一致性

1.3.2 实现难点与挑战

反转节奏是最难控制的一件事。游戏的核心是三重身份的逐步揭露：先让玩家确认所谓闹鬼是人为的，再确认主角就是设计这套装置的王阙，接着确认王阙就是十七年前失踪的陈晋年，最后确认匿名举报的白灯客也是同一个人。四次揭示必须按固定顺序发生，任何一次提前泄底，后面几步就全部作废。大纲评审时反复调整过村民的台词，把原本多人直接叫出主角名字的写法删掉，只留一个记忆混乱的老人偶尔喊出陈晋年的名字，多数村民必须明确认为陈晋年已经死了。

故事逻辑的自洽花了比预想更多的时间。大纲修订清单里列出的问题大多属于这一类：搜救队第二天就找到两个孩子，和陈晋年昏迷两天后才被采药人发现，这两件事不能同时成立；妹妹事故的逐日时间线前后指代不清；陈晋年在早期稿子

里被写成冻结在十七岁，但他其实是十七岁的兄长，坠入采坑的是妹妹。父亲的动机也重写过，最初只写他远远看见白布却不敢确认，心理因果太弱，改成了他既受过胁迫、也主动毁过证，这样他后来十七年不寻找可能生还的儿子才站得住。

技术上的主要矛盾是状态一致性。剧情、探索、对话、背包、成就各自都有进度要记，如果每个模块各存一份，迟早会出现同一次操作在不同模块里结果不一致的情况。V2 的解决办法是收敛到单一进度源，并规定新代码不得读写兼容字段来决定剧情推进。协作层面同样靠约定约束：接口先给真实输入输出，双方共同跑通才算接入；页面结构、剧情文本、热点规则、场景美术、状态与存档分别归属到人，问题按归属回退；开发者不能做自己 PR 的唯一审查人。

还有一类困难来自恐怖规则要同时是玩法规则这条要求。白灯、广播、湿脚印和墙上姓名都成立，但每种装置都必须有供电范围、触发条件和可以被玩家反向利用的缺陷，否则操作者就显得无所不能，恐怖也就失去了张力。拆鬼那一段的设计正是围绕这条原则展开的：玩家要逆向追踪三种异象的来源，利用延迟或供电盲区躲过下一次触发。

团队的每一位成员都在这个项目中投入了超出课业要求的心力。我们反复推敲每一句对白的分寸、每一个反转的节奏、每一条命运线索的闭环，因为我们确实相信这个故事值得被好好讲出来。我们未必做到了完美，但我们做到了诚实。如果玩家在走出这个故事之后，能对沉默和逃避多一分警觉，对面对多一分勇气，那便是我们最大的回馈。

二、游戏设计详解

2.1 主线剧情

2.1.1 开场与物品

玩家从祠堂的地上醒来，身上只有三样东西：一张烧毁大半、只剩姓氏的工作证，一颗蓝玻璃珠，一把没有标记的老钥匙。守在一旁的年轻人自称小周，告诉他你是公司外聘的逻辑分析师，来调查村里的闹鬼事件，前几天摔伤了头。

这句话只有一半是真的，而玩家此刻无从判断是哪一半。**失忆是整套剧情的启动装置：玩家与主角站在完全相同的信息起点上，村民对同一张脸给出互相矛盾的称呼，有人叫他陈晋年，多数人只认识来谈拆迁的王工程师。**玩家要回答的第一个问题因此不是村里有没有鬼，而是我是谁。

三件开场物品各自是一条身份线的入口。工作证把玩家带向项目档案室，钥匙打开陈家老宅的门，蓝玻璃珠后来在邻镇旧诊所的登记抽屉里找到同款。三条线由身上的物件自然牵引出来，玩家不会觉得自己在完成一份任务清单。

2.1.2 三条身份线

剧情采用一个村庄中枢、三条身份支线、两次强制汇合、一个矿井终局的半线性结构。村口是唯一的中枢，玩家在这里反复出入，把三条线的进度带回同一个地方。

三条线各自回答一个问题。陈晋年线回答我从哪里来，经由旧家、坠沟处、空坟和诊所档案，确认十七年前的兄妹坠沟与父亲的签字。王阙线回答我做过什么，经由村委旧楼、项目部、矿区控制室，确认恐吓系统与封井任务出自谁手。白灯客线回答我删掉了什么，经由拒签户收到的匿名信、废弃邮电所和匿名藏点，确认那份举报在哪些地方被有意留白。

三条线可以交换探索顺序，但交汇点固定。第一次汇合把所有灵异现象的物理成因摊开，第二次汇合还原苏禾之死。玩家在两次汇合之间获得的不只是线索，还有一份越来越难维持的自我解释。

2.1.3 五次揭示顺序

全篇的悬念由五次揭示推进，它们必须按固定顺序发生。任何一次提前泄底，后面几步会一起作废。



图 2.1 剧情结构总览

1. 借灯现象可以人为制造。
2. 主角就是恐吓系统的设计者王阙。
3. 王阙正是十七年前失踪的陈晋年。
4. 白灯客也是主角，而且他的举报刻意避开了自己。
5. 小周把苏禾推入井中之后，王阙明知她还活着，仍然亲口选择了不报警。

前四次是对玩家认知的推翻，最后一次把矛头转向主角的道德选择。至此玩家已经知道自己操控的这双手做过什么，剩下的问题与推理无关，只与意愿有关。

为了让这个顺序守得住，村民的台词和档案的可辨识度都做了限制：多数人必须认定陈晋年已经死了，只有一位记忆混乱的老人会偶尔喊出这个名字；王阙的正脸资料在档案里始终被剪去；白灯客从不出面，只以变声录音、匿名信和统一的灯形印记存在。

2.1.4 五结局设计

终局不给玩家一句立场选项，而是让玩家实际处置手中的证据材料。交出证据、追逐失败、销毁全部、选择性公开、完整公开，五条路径通向五个稳定结局。

销毁全部，村庄在官方记录里继续什么也没发生。选择性公开，玩家保留矿难与集团违法的部分，删掉王阙设计恐吓、拒绝报警和白灯客删证的记录，以揭密者的身份获利，完成又一次责任切割。完整公开，四组材料一起放出去，主角失去所有干净的身份，第一次以同一个名字承担全部选择。

此设计的用意在于，三个结局的差别不体现在态度，而体现在玩家愿意删掉什么。到这一步，玩家做过的事和主角做过的事是同一件事，这也是项目的核心设定

2.2 探索场景

2.2.1 场景即界面

探索部分放弃列表式菜单，改用整张场景图作为交互载体。全篇共十六余个场景，覆盖村庄中枢、三条身份线、三处玩法场景和五个结局画面。

场景按统一规范绘制与接入，画布 1280×720，16:9，满屏不裁切，图片与热点同步缩放；可调查的对象一律做成场景里的热点，不做平铺网格；单个点击目标不小于 44×44 像素，热点中心间距不小于 144 像素，保证在 390 像素宽的窄屏上依然点准；同一台面超过三个热点时，改为一级热点加放大层。

在此规则约束下，玩家在场景里看到的每一样东西，都可点、可问、可拿，场景美术因此不能只是背景板，它同时是线索的容器。

场景中的对象在被调查后会改变状态，或在视觉上留下痕迹，或标记为已调查，玩家不会重复拿到同一条线索，也不会有漏掉某个对象就必须重开一局的焦虑。

2.2.2 场景清单

表 2.1 探索场景清单

场景	所属线路	主要交互内容
祠堂	序章	检查伤口与三件随身物品，在白灯出现后寻找出口
村口	中枢	向村民打听，记录彼此冲突的说法，取得村庄手绘图
山腰旧运输道	陈晋年线	按旧送葬路线寻找坠沟处，判断两人滚向不同深度
陈家老宅	陈晋年线	用刻名钥匙开门，核对照片、校服、刻痕与送葬名单
衣冠冢	陈晋年线	确认墓中没有遗体，找到邻镇诊所的票据编号
村委旧楼	王阙线	恢复档案室电脑，查看拆迁名单、恐吓执行表与被遮去姓名的背景调查

接下页

(续表 2.1)

场景	所属线路	主要交互内容
临时项目部	王阙线	读秘密封井任务单，看到旧井写成未登记安全隐患
废弃小学	苏禾线	还原她对电线、广播与排水设施的巡查
旧矿图藏点	苏禾线	取出父辈证词、原始矿图复印件与封井材料
废弃邮电所	白灯客线	收集灯形印记的条款提醒、录音与匿名举报草稿
矿区旧控制室	王阙线	拼合系统图纸，读取调试日志与低清录像
邻镇旧诊所	陈晋年线	用票据编号查到账簿、伤情图与无名病人转诊记录
父亲旧屋	父亲线	进入暗层，找到爆破日志、威胁材料与未寄出的证词
匿名藏点	白灯客线	开启藏点，比对原始文件、举报版本与删除记录
广播室	玩法	拆鬼小游戏，逆向追踪相关恐怖故事的来源
旧矿井	玩法	路线解谜小游戏，绕过封堵进入竖井
数据机房	玩法 / 终局	对峙追逐小游戏，取得记录并决定证据处置方式
村口白灯	结局	五个结局

2.2.3 人物对话

不同角色说不同的话是核心塑造规则。小周冷静克制，从不用感叹句，也很少直接否定主角，在关心主角的同时显露出一些异常；村民保留口语和方言味，同一件事各有各的说法；主角的内心独白短，且多数以疑问收尾。

信息有分寸。村民群体被拆成有差别的记忆持有者：拒签户、已经搬走的人、当年收过补偿的人、仍然相信借灯禁忌的老人，对十七年前的事故各有各的版本。玩家在村口收集到的说法互相冲突，冲突本身就被设计成线索。

2.2.4 背包与线索

调查得到的物证进入背包，线索独立成页，两者分开管理。玩家拿到的是物证，理解的是线索，证据链由线索之间互相印证构成。

三条身份线各自提供的证据能证明的事情是有限的，设计时刻意保留了这一点。陈晋年线的照片、票据与空坟记录能证明兄妹坠沟和陈晋年生还，证明不了集团的违法；王阙线的背景调查与调试录像能证明恐吓系统出自谁手，证明不了他事前知道旧事故的真相；白灯客线的匿名信与删改记录能证明泄密时回避了自身责任，不能等同于完整自首。任何一条线单独拿出来都推翻不了全部假设，玩家必须把三条线的证据拼在一起。

2.3 核心交互

2.3.1 剧情对话

旁白、系统提示和人物对白统一走阅读组件。组件负责逐段显示、控制继续与关闭；剧情模块决定说什么、能做什么；对话模块负责具体的角色对白。三层分开之后，改一句台词不需要动流程，加一个流程节点也不需要重写文案。

对话框在窄屏下换行重排，发言者名牌与正文分行渲染，多行文本不会顶掉名牌。人物立绘按情绪切换，配合字号与留白的变化，让长时间阅读不至于疲劳。

V2 之后游戏不设影响主线的分支选项。玩家只在剧情明确给出操作时才能提交选择，普通的继续阅读不会改变任何状态。这条限制是刻意的：故事要的是玩家沿着一条确定的因果链走到底，而不是在岔路口反复权衡。

2.3.2 四种小游戏

小游戏嵌在剧情节点上，玩完把结果回写给剧情模块，由剧情模块决定后续走向。四个小游戏分别承担不同的叙事功能。

手绘地图复原放在村口，玩家从三名村民处取得地图碎片后拼成完整村庄图，拼好即解锁通往陈家老宅的路径。它教玩家读懂这个村庄的空间关系。

拆鬼安排在第一次汇合。玩家要逆向追踪童谣、白灯和湿脚印的来源，关闭其中一组装置。每种装置都有供电范围、触发条件和可以被反向利用的缺陷，玩家利用延迟或供电盲区躲过下一次触发。恐怖规则在这里正式成为玩法规则，也是全篇唯一一处让玩家主动拆解恐惧的地方。

路线解谜发生在旧矿井。玩家依照原始矿图从排水洞绕开封堵，检查两道挡墙、废石填层和速凝浆的痕迹。这一段的目的是让玩家亲手确认一件事：当晚封掉的只是唯一公开入口，竖井从来没有被灌满，井下的人本可以被救出来。

结局追逐放在数据机房，详见本章第四节。

四个小游戏都不以难度取胜。它们是为让玩家在关键节点上动手，而不是让玩家卡关。

2.3.3 恐怖表现与设计

恐怖表现集中在三类手法上，都服务于同一件事——让玩家相信有什么东西在这个村子里。

音效是其中最重要的一层。檐下的雨声和环境底噪持续存在，广播里的童谣带着电流杂音定时出现，远处传来爆破的回声。这些声音同时属于村庄的日常和恐吓装置的一部分，玩家在中段之前无法分辨，中段之后回头看，会发现每一处声音都解释得通。

画面层面用黑屏、花屏和抖动处理关键转场，全篇共十余处。它们的共同规则是只出现在状态切换的瞬间，用来制造一次短暂的失控感，不侵入阅读区域，不影响正在显示的文本。祠堂断电那一段是其中最典型的一处，灯灭与转场叠在一起。

过场视频用在几个无法用静态场景交代的位置，配合白灯 CG 与画面闪黑衔接前后场景。人物的情绪转折、时间的跳跃、回忆片段的插入都交给视频处理，避免用大段旁白去解释画面。

所有恐怖手法的边界是明确的：不打断操作，不遮挡文本，不靠突然的巨响吓人。真正的压力来自玩家逐渐意识到自己拆解的每一个装置都眼熟。

2.3.4 存档与进度恢复

游戏状态保存在浏览器本地，包含剧情检查点、已确认事实、调查进度、对话进度、物品、线索、地点、地图拼图结果、选择和成就。

剧情检查点是唯一可信的进度来源。页面、探索、对话和小游戏都不能自行修改当前节点，一次操作只有在状态提交和存档成功之后，才会在界面上显示为完成。这样做的收益在协作阶段体现得最明显：五个人分头开发，谁改动一个模块都不会让另一处的进度对不上。

注册用户和游客使用各自独立的本地存储空间，互不覆盖。系统分别处理没有存档、存档损坏、版本不兼容和浏览器无法写入四种情况。恢复失败时保留原始数据并如实告知玩家，不会悄悄当成一局新游戏重新开始。

2.4 附加功能

2.4.1 终局追逐与生命值系统

设计的终局小游戏是数据机房主题追逐关卡，玩家需躲避小周追捕，收集三份关键证据并撤离出口。关卡引入生命值系统，强化追逐生存压力；受击后提供 1 秒无敌时间，防止连续碰撞瞬间暴毙，平衡游玩容错。对斜向移动做速度归一化处理，走位操控更加稳定流畅。证据具备防重复计数逻辑，关卡成败结果可回传主线，联动整体剧情。俯视视角角色立绘搭配密闭机房场景，便于观察敌人动向，进一步烘托出悬疑紧迫的逃亡氛围。

2.4.2 成就系统

成就页共收录 14 条成就，按剧情推进的层次分成三组：调查札记 3 条，灯下真相 6 条，终局余烬 5 条。页面标题是雨夜留痕，顶部给出解锁进度与一条进度条，下面是三组成就卡片。

每条成就的解锁条件不是另写一套判定，而是直接声明它依赖哪些已经登记的剧情事实。地图复原完成对应残图归一，确认异象系人为制造对应灯下无人，三重身份合并对应三名照一身，终局追逐生还对应雨幕留痕。成就因此不需要维护第二份进度，剧情走到哪里，成就就跟到哪里。

表 2.2 成就目录

分组	成就	解锁条件
调查札记	残图归一	完成村口手绘地图复原
	尘封仍有回声	走完陈家老宅一段并确认前后说法矛盾
	雨线交汇	四条外围调查线全部走完
灯下真相	灯下无人	确认白灯与童谣系人为布置
	三名照一身	陈晋年、王阙、白灯客三条线合并
	封井之外	绕过灌浆封堵进入旧矿道
	迟来的证词	还原苏禾之死的完整链条
	灯尽见真章	四组证据全部集齐
	雨幕留痕	在终局追逐中生还
终局余烬	灯火照旧	结局一 · 交出证据
	井口又高一寸	结局二 · 追逐失败
	姓名落尽	结局三 · 销毁全部证据
	借来的黎明	结局四 · 选择性公开
	此后无须借灯	结局五 · 完整公开

五条结局成就被标记为隐藏条目。未解锁时，成就页只显示未明之章配合罗马数字序号，描述文案完全不渲染。这条规则是为了防住一个很容易踩的坑：成就页是最容易被玩家提前翻看的地方，如果把五个结局的名字直接列出来，整条主线的走向就白送了。成就页因此对未解锁的结局保持沉默。

卡片左侧用一个字区分状态，已解锁显示灯，未解锁显示影。规则模块判定出某条成就可以解锁，不等于它已经解锁——事件要由状态模块按游戏局、存储范围和幂等键提交成功，卡片才会点亮。同一份成就模块只允许一个视图负责新解锁提示，避免同一件事被提示两次。

数据异常被限制在单条记录内。某条成就缺少对应剧情事实，或解锁时间格式无效，只有该条显示记录异常，其余卡片和整个页面的生命周期不受影响。读档失败也按原因分别处理：存档损坏、版本不兼容、登录身份与存档不匹配、浏览器存储不可用各有各的提示；没有存档不算失败，页面会显示当前还没有游戏存档

2.4.3 项目组信息

制作组与项目介绍分成两个页面，五名成员另各有独立页面，页面之间互相链接。制作组页面的内容分三块：团队概览给出组名白灯客、项目名与版本号，以及一句团队自述；成员档案列出五人的头像、角色与一句话简介；项目分工按版本记录每个人做过什么。

这些资料全部集中在一个数据文件里，HTML 只管渲染，不在页面里硬编码成员信息。这条约定的好处在改稿时最明显：调整某位成员的分工或换一张头像，只需要改数据文件的一个字段，五个页面同时生效，不会出现某个页面还是旧版本的尴尬。

成员页面同样按数据驱动的方式渲染。每页有档案编号、姓名缩写标记、头像、角色、一句话简介、个人自述与个人引语。字段为空时不显示，而不是显示待补充的占位文案——这条原则写在了数据文件的文件头，成员没有提供的资料，页面就空着，不替他编。头像加载失败时该图片元素直接移除，不会在页面上留下一个破图。

表 2.3 项目分工（按版本）

成员	各版本分工
罗晨菲	V1 全局、主菜单、页面导航与存档；V2 全局状态、存档、错误处理与正式集成，需求文档撰写；V4 全局状态协调，素材、视频、音效接入
杨梦	V1 入口与核心视觉；V2 场景美术、素材与视觉验收；V3 场景美术、视觉、音效与剧情回归；V4 转场素材、剧情视频、音频素材及接入
高冰轩	V1 小游戏、项目介绍与制作组页面；V2 游戏页面、统一阅读组件与响应式；V3 全局协调、状态存档；V4 三个小游戏的设计与完善
卢正松	V1 场景探索、NPC 对话、背包与成就；V2 热点、对话、线索、背包与成就；V3 点击式探索与整体功能集成；V4 对话框设计、物品详情卡片与菜单页调整
于知让	V1 账户与剧情模块，需求文档撰写，剧情初稿确认；V2 剧情 Node、剧情展示与剧情回归；V3 需求文档撰写，V4 剧情提示与 NPC 对话完善，文字悬疑深入

分工按版本而不是按模块记录，是因为这个项目里每个人的职责一直在换。五个人在四个版本中轮换承担项目经理、产品经理、技术负责人等职责，同一个人不同版本里做的可能完全不是一件事。按版本逐条写下来，谁在哪一代交付了什么就一眼可见，期末回看也不会把两个版本的工作混在一起。

2.4.4 音效与美术

音效与美术围绕同一套听觉与视觉规范展开。场景美术的配色限定在冷夜、连阴雨、雾和白灯暖光四个方向上，白灯是全篇唯一使用冷青高光的物体，其他光源一律压低。这套限制让玩家在昏暗的场景里始终能一眼找到该点的东西，也让白灯在视觉上始终是例外。

音效负责环境层，美术负责信息层，两者共同的底线是不干扰阅读。具体的视听规范、色彩体系与界面设计见第三章。

这一章写的都是我们最终要做成的东西。四个版本迭代下来，我们把故事结构、场景规则、交互层级和视听手法全部拆开重排过不下五次，为的是让玩家在三个小时之后，回头能找到每一个细节的出处。我们未必把每一项都做到了最好，但每一项为什么这么做，每个人都很清楚。

三、美学与用户体验

3.1 视觉设计规范

3.1.1 整体风格

《借灯》的视觉只讲一个对比：冷夜里的那盏灯，基于强烈的冷暖对比进行设计。

故事里的白灯笼没有火焰，却在雨夜里泛着白光。它代表掩饰，在画面中被做成全篇唯一的暖光源。其余元素，雨、雾、废弃的矿区、没有窗的屋子，全部压在冷调里。玩家进入任何一个场景，视线都会先落到亮的那一点上。

这个对比同时解决了一个实际问题。全篇场景出自不同批次，祠堂在室内，矿井在地下，数据机房是现代的，如果各画各的，玩家每换一个地方都要重新适应。规范因此固定四样不变的东西：色板、雨、灯、字体。场景内容可以不同，这四样必须一致，玩家切换场景时感受到的是同一个雨夜里的不同地点。

3.1.2 配色

全站颜色收拢到一个变量文件里，其余样式表只引用变量名，不单独书写色值。这样做的目的是让配色一致性自动维持：修改一处底色，所有页面同步变化，不会出现主菜单与老宅场景相差三个色阶的情况。

用色规则可以概括成一句：底色永远冷，强调永远暖。

朱砂红是使用限制最严的颜色，只允许出现在三处：关键真相线索、错误提示、门外那个叫名字的声音。一旦大面积使用，它就不再表示紧急，玩家对它的警觉也会随之消失。

两项可读性指标定为硬性要求：正文与背景的对比度不低于 4.5:1，最小的楷体字不小于 12px。探索区叠加在场景图上的标签必须垫一层深色半透明底，不能把浅色字直接压在亮背景上。场景图中存在高亮区域，浅色标签叠加其上会失去可读性。

3.1.3 字体

宋体笔画清晰，适合长段落阅读，用于剧情正文和场景名。楷体接近手写，放在说明性小字上，与正文形成区分，避免提示文字打断阅读。按钮和顶栏需要快速识别，使用无衬线字体。

表 3.1 全站色板

分组	色块与 Token	用途
冷底	--bg (#07090e)	页面最深底色，场景最暗处
	--bg-elevated (#0d1118)	面板与卡片底
	--panel	文本框与侧栏的半透明底
纸与暖金	--lamp (#f5f0e3)	白灯的光，热点光晕与高光
	--paper (#ece7dc)	正文纸白
	--paper-dim (#cfc8b8)	次级文字、旧纸
	--accent (#d8c6a4)	强调暖金，木纹、旧物、描边
	--accent-deep (#b89b6e)	深金，按钮悬停态
冷点缀	--mist (#8fb8ce)	雾蓝，冷光与次级点缀
	--rain (#5d7f9e)	雨蓝，雨丝与冷调
警示	--cinnabar (#a6534a)	朱砂红，关键真相与门外的声音
	--danger (#c17a74)	错误与警示

表 3.2 字体与字号阶梯

层级	字体	字号 / 行高	用途
标题	宋体	24px / 字距 0.18em	游戏名、章节名、场景名
正文	宋体	17–21px / 1.9	剧情正文、人物对白
旁白与小字	楷体	12–15px / 1.6–1.8	说明性小字、旁白细节
界面标签	无衬线	14px	按钮、顶栏、系统提示

封面单独使用一套毛笔手写字体，全篇只在封面出现一次，重复使用会削弱它的辨识作用。

正文的 1.9 倍行高和标题 0.18em 的字距是依据中文排版特点设定的。中文没有词间空格，字号越大、字距越密，长句越容易读串行，行高和字距是替代空格的主要分隔手段。

3.1.4 人物立绘

立绘只有两个站位，左和右，分别落在画面两侧 5% 的位置。哪个角色站哪边写在数据里，渲染层只读站位数值，不判断来的是谁。这样加一个人物不需要改渲

染代码，改一个字段就够了。

立绘的作用是表示场景中有人在场，同一时刻只出现一个，站位的变化即表示说话人更换。

苏禾全篇不出正脸立绘，只以笔迹、照片和登记记录出现。这是因为，她的死要等到第二次汇合才揭晓，任何一张正脸立绘都会让玩家在她真正出场之前就先认出她。美术不必把每个角色都画出来，隐藏不该出现的角色同样是设计的一部分。

3.1.5 美术原则

场景图同时承担背景和线索载体的功能，美术有四条硬性要求。

1. **光源统一。**每个场景只设一处主白灯光源，位置随场景走：祠堂在门和梁上，村口在电杆和屋檐，老宅在门后。光晕半径约为场景短边的一半到七成。任何情况下不得遮住发光热点、线索和对白区。
2. **冷暖不混。**关键对象和热点不许埋在暖光晕里。玩家点哪一处、就看哪一处
3. **画布与安全区统一。**场景按 1280×720 绘制，16:9，满屏不裁切。底部 600 到 720 一条留给阅读框和按钮，关键信息不往里放。
4. **背景不许剧透。**每张图交付前要过一遍：这张图有没有提前泄露某个人的结局、某一层身份的关系。有就改。

这四条写进了场景交付模板，每张图由美术、剧情和前端三方逐项确认之后才能合入。氛围统一因此有了可以逐条核对的验收标准。

3.1.6 动效规范

日常动效控制到几乎不可察觉，只有三类。

雨丝垂直缓慢下移，透明度很低，不干扰阅读。白灯的光晕在 0.85 到 1 之间以约 2.5 秒的周期缓慢起伏，该效果只用在灯和发光热点上。场景切换统一使用 200 到 400 毫秒的淡入淡出，全篇不做变化。

限制强度的原因是：玩家要在这个界面上读三个小时的字，任何一处抢注意力的动效都会被反复看到几百遍。动效的作用是维持画面的活性，不是吸引玩家注意动效本身。

规范中另有一条硬性要求：动效不触发眩晕，不做大面积快速位移，并且尊重系统的减少动态效果设置。高强度的动态效果全部留给关键转场，见本章第四节。

3.2 文字风格

3.2.1 整体调性

全篇有一条硬性要求：游戏里出现的所有文字都要出自故事世界内部，不出现项目组直接对玩家说话的情况。

这一条在执行时比想象中容易破。V4 定稿时删改幅度最大的一批文案，问题几乎都出在这里。原文里有一句“恭喜你，成功解锁【陈家老宅】相关剧情”，句式上是系统提示，效果上把玩家从故事里拽了出来，让他在沉浸最深的时候意识到自己在操作一款游戏。同一批改写里还有一处“归档结论”，用文件摘要的语气替玩家把结论下完了。这两处最后都换掉了：前一句整句删除，后一句改由主角的独白提出，而且停在问句上。

判断标准可以总结成一句：结论由玩家得出，文本不替玩家下结论。

3.2.2 用语

写作时要求描述现场能直接看到、听到的东西。

改稿前有一段描述证据吻合的文案，用的是重合、证明、表明这一类词。这是文件摘要的口气，读起来像在审一份报告。改写之后变成了：纸已经很旧了，可撕口还是白的，是最近才撕开的。两句话说的是同一个判断，前一种写法要玩家自己翻译一遍，后一种写法直接把画面放到他眼前。

抽象概括会让玩家觉得自己在读材料，具体的形貌、声音、温度，才会让他觉得自己站在那个地方。

角色语言的个体差异在第二章已经展开过，这里只补充它对叙述层的要求：同一件事，村民用口语，小周用短句，主角用疑问句。叙述层不跟随任何一个人的语气，它始终是全篇最冷的那一层。

3.2.3 氛围营造

氛围的营造依靠信息的分寸，不依靠形容词。

全篇没有一句直接告诉玩家这里很吓人。与其写雨夜的村子很阴森，不如写纸上那道撕口还是白的。先给出一个具体的东西，再把它留给玩家自己去理解。

名牌制度也服务于这一点。旁白不出名牌，正文自己陈述，玩家读到的是一段没有来源的叙述；独白出名牌，那句话就被标记为主角自己的声音。两者在视觉上区分得很清楚，玩家始终知道一句话是谁说的，或者知道它没有来源。

情节里最沉重的细节，我们选择不做解释。父亲签下停止搜救那一段，处理方式是让玩家自己停下来，文本不跟任何评价。这种留白是刻意的，目的是把最后一步留给玩家自己完成。

3.3 界面设计

3.3.1 站点地图

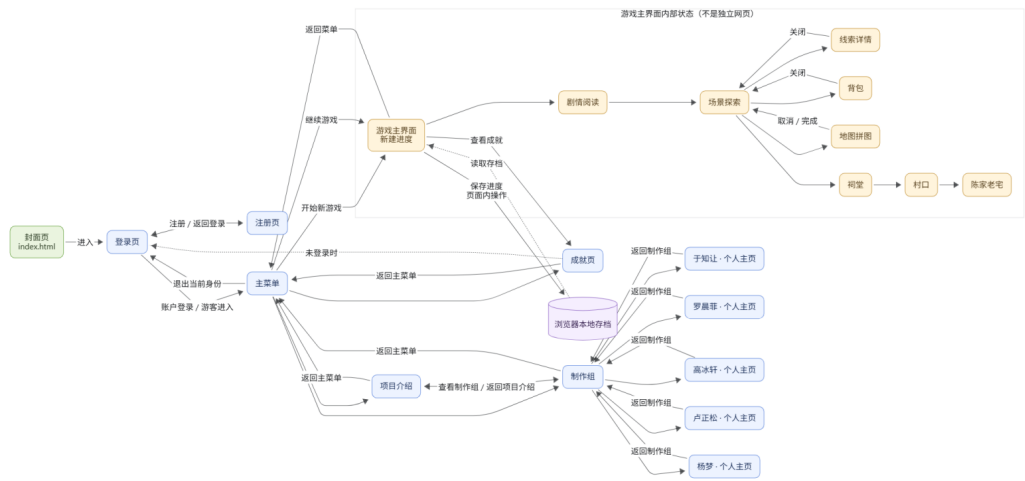


图 3.1 站点地图结构

3.3.2 线框图

【待补充】

3.3.3 设计图



图 3.2 祠堂灯亮设计



图 3.3 四条探索分支背景设计

3.4 恐怖美学设计

3.4.1 整体特点

全篇的恐怖不使用惊吓手法。

惊吓的效果只持续几秒。我们采用的方式是持续的心理压抑：玩家玩到中段时意识到，自己刚才拆开的每一个装置，材质、颜色、声音，都来自主角小时候见过的东西。那一刻不产生惊吓反应，产生的是心理上的沉重感。

因此全篇没有突然出现的巨响，主要手法都带有一层铺垫：灯先闪两下才暗下去，声音先响画面再跟上，门外先叫一声名字，隔一拍玩家才听清那是谁。恐惧感来自预期落空的那半秒，与音量无关。

这套做法的核心是反差。玩家在前半段接受的是一套民俗禁忌的解释，中段被告知所有异象都是人为布置，此时本该放松。紧接着他会认出，布置所用的元素全部取材于主角的童年创伤，他把自己最怕的东西做成了恐吓他人的工具。恐惧的来源由外部转向人本身，并且落在玩家自己操控的这双手上。

3.4.2 音效

音效承担环境层，同时负责制造真假难辨的效果。

檐下的雨、排水渠里的水、广播里带电流杂音的童谣、远处传来的爆破回声，这几层声音持续存在。它们同时属于村庄的日常环境与恐吓装置的一部分。玩家在中段之前分不出哪一层是人为布置的，中段之后回头看，每一声都解释得通。让人不安的是玩家发现自己听了两个小时，却始终没有听出其中的区别。

背景音乐只有一条循环轨，音量控制在四成左右，切换页面时不重新起播。玩

家在菜单、成就页和游戏之间来回切换，音乐不应中断。

浏览器的自动播放限制决定了第一次出声必须等待玩家的第一个操作。音频在玩家第一次点击或按键之后起播，在此之前保持静音，界面上另有一个常驻的静音开关。这项限制无法绕过，可以做的是不让它成为意外：玩家按下第一次鼠标时音乐正好响起，让它看起来是设计好的。

音频加载失败不能连带停止游戏。播放器返回失败码后，游戏降级为无声继续，控制台保留一条记录。玩家可以听不到背景音，但不能卡在黑屏前。

3.4.3 视频

过场视频用于静态画面无法交代的位置：人物的情绪转折、时间的跳跃、以及回忆片段的插入。这些位置用旁白叙述会拖慢节奏，因此交由视频承担。

全篇七段，序幕两段，五个结局各一段。

表 3.3 过场视频清单

触发位置	素材	内容
序幕 · 白灯初现	light_up	门外忽然亮起一盏灯
序幕 · 祠堂断电	cut-off	祠堂内忽然断电
结局一 · 交出证据	the_ending	灯火照旧
结局二 · 追逐失败	close_well	井口又高一寸
结局三 · 销毁全部	unnamed	姓名落尽
结局四 · 选择性公开	after_whitelamp	借来的黎明
结局五 · 完整公开	no_more_lamp	此后无须借灯

触发条件绑定在台词内容上，不绑定在剧情节点的编号上，读到对应句子才播放。这样处理在协作阶段省去很多返工：节点编号会随剧情改写而变动，如果视频绑定编号，每改一次文案就要重新核对七个编号，漏掉一个就会导致一段视频永远不播放。绑定文本之后，修改剧情不会漏掉视频，增加视频也不用改动剧情数据。每条视频只播一次。

3.4.4 转场

转场是全篇唯一使用高强度效果的位置。日常动效控制到几乎不可察觉，效果强度集中在下面这十一处。

三类基本手法，加一种抖动接闪黑的组合手法，共同点是都带有铺垫，没有一处是突然出现的。

表 3.4 转场手法

手法	时长	过程
轻闪黑	0.9 s	快速压黑，短暂停顿后淡出
窒息式闪黑	3.3 s	灯闪两下 → 黑幕以 550 ms 漫上 → 黑暗中边缘红晕呼吸 1 s → 以 1.5 s 缓慢退出
暴力抖动	0.65 s	位移从 18 px 逐级递减到 0，叠加旋转与轻微模糊
无信号花屏	2.1 s	随机噪点从画面某一点扩散到全屏，短暂停顿后收回到同一点并淡出
抖动接黑	3.1 s	先抖动，120 ms 后黑幕漫上，其余同窒息式

窒息式闪黑中的两次灯光闪烁有明确作用。先铺垫再压黑，玩家在黑暗真正到来之前已有半秒的预感，恐惧感来自这半秒的预期落空，与黑暗本身无关。恢复阶段用 1.5 秒，比压黑慢了三倍，目的是让玩家在离开黑暗时多停留一会儿。

花屏的扩散和收束都从一个随机点开始，两次使用同一个点。收束速度慢于扩散，形成缩回原处的效果。

十一处触发同样全部绑定在台词上。一个监听器观察剧情文本的渲染结果，读到约定的句子就播放对应效果，剧情数据里不埋任何标记。这样处理的好处与视频一致：修改剧情不会漏掉转场，增加转场不需要改动剧情。

所有手法都绘制在场景层内部，与阅读区是两个独立的层。黑屏期间文本仍然存在，只是被遮住，恢复之后玩家可以接着读。全篇没有一处转场会顶掉正在显示的文字，也没有一处会接管鼠标。恐怖效果可以打断节奏，不中断操作。

四、技术方案与质量保证

4.1 开发与运行平台

本项目是纯前端静态站点，全部代码在浏览器端运行，不依赖服务器进程与数据库。平台选择与三周的开发周期有关：成员使用的操作系统并不一致，项目因此不能依赖任何平台特有的工具链；没有服务端与构建步骤，任何人克隆仓库之后可以直接打开页面验证效果。

表 4.1 开发与运行平台

项目	方案
开发平台	常规代码编辑器、Git、GitHub；成员可使用 Windows、macOS 或 Linux
前端技术	原生 HTML5、CSS3、JavaScript，不引入没有实际需要的框架和打包器
运行方式	静态站点，通过本地 HTTP 服务或部署后的站点地址访问
运行平台	最新版 Chrome、Edge；桌面端与移动端浏览器视口
数据保存	浏览器 localStorage，注册用户与游客使用不同存储键
版本管理	GitHub Flow，main 为唯一长期分支，功能在短期 Feature 分支开发

4.2 技术选型

4.2.1 原生技术栈

本项目不使用框架和打包器。组内成员的编程基础不一致，原生 HTML、CSS 和 JavaScript 可以让所有人立即投入开发，省去框架与配套工具链的学习成本。调试链路也是最短的：改完代码刷新浏览器就能看到结果，没有编译等待。轮值交接时，接手的人不需要先理解框架的生命周期或响应式机制。

代价是没有现成的组件复用与路由方案，这部分需要自行实现。本项目页面数量有限，全局状态集中在存档与进度上，自建的成本低于引入框架的成本。

4.2.2 浏览器本地存储

玩家进度保存在浏览器的 localStorage 中，不经过服务端。

存档键的格式为 `white-lamp:save:{身份}:v2`。身份段有两种取值，游客为 `guest`，注册用户为 `account`：加账户 ID。两种身份的数据物理隔离，游客存档不

会覆盖注册用户的存档。键名中带存档格式版本号，格式升级时不覆盖旧数据：读到旧版本的存档会提示玩家开始新游戏，旧档保留在原处。

账户信息另存于两个键，前缀同为 `white-lamp:auth:`，其后分别是账户表 `accounts:v1` 与当前会话 `session:v1`。

`localStorage` 的容量上限为每个站点 5 至 10 MB。本项目存档只记录进度、已读文本与成就状态，体积远低于这一上限。

4.3 技术架构

代码按表现层、逻辑层、数据层三层组织，三层之外有一个全局状态对象作为数据中枢。



图 4.1 技术架构图

表现层是 HTML 与 CSS。页面使用语义化标签，样式分为全局、页面、游戏三类；全站色值与间距收在 `tokens.css` 一份变量表里，其余样式表只引用变量名，不写死具体色值。

逻辑层是原生 JavaScript，按功能域拆成七个模块：核心、主线、探索、小游戏、账户、成就与介绍页。模块之间的调用关系是单向的，主线、探索与小游戏各自读写全局状态，彼此不直接引用。改动其中任一模块，不需要通知另外两个。

数据层是浏览器本地存储，读写统一经过 `core/storage.js`，带版本校验与失败兜底。剧情内容不放在这一层，它是随代码一起发布的静态数据，不写入玩家设备。

4.4 代码组织

目录结构的设计目标是让任何人打开仓库都能迅速定位到目标文件。四条规则：一个功能放在一个明确位置；页面数量可数；资源按类型分层，并在二级目录下继续按功能细分；数据与逻辑分离，全部剧情内容放在 `game-line/data/` 下，引擎只负责渲染和推进，不硬编码具体对白。新增剧情阶段时只增加数据文件，引擎核心代码不动。

Listing 4.1 项目目录结构

```

1 jiedeng/
2 +-- index.html          cover page
3 +-- pages/              12 game pages
4 |   +-- game.html       main story
5 |   +-- menu.html       main menu
6 |   +-- authorize/      login / register
7 |   +-- achievements/   achievements
8 |   \-- about/          project, team, 5 member pages
9 +-- assets/
10 |   +-- css/
11 |   |   +-- tokens.css  design tokens (single source)
12 |   |   \-- ...         per-page stylesheets
13 |   +-- js/
14 |   |   +-- core/       global state, save, nav, transitions, audio
15 |   |   +-- game-line/  story engine; data/ holds all story data
16 |   |   +-- exploration/ hotspots, items, branching dialogue
17 |   |   +-- minigames/  4 minigames + shared 2D engine
18 |   |   +-- authorize/  account, session, permission
19 |   |   +-- achievements/ definitions and checks
20 |   |   \-- about/
21 |   +-- images/         scenes, characters, items, UI
22 |   +-- audio/          background music
23 |   +-- videos/         7 cutscene videos
24 |   \-- fonts/
25 +-- test/               unit tests and browser regression
26 \-- docs/               requirements, interfaces, story, this report

```

各模块的规模如表 4.2。

样式表另有 12 个文件、3,802 行，其中 `tokens.css` 是全站唯一的变量来源。图片资源 86 个，按用途分为场景图、人物立绘、物品与界面四类；过场视频 7 段，

表 4.2 代码模块划分与规模

模块	文件	行数	职责
core	13	4167	全局状态、存档、页面控制、导航、过场与音频
game-line	13	4126	剧情引擎与全部剧情数据
exploration	22	2545	场景热点、物品与背包、分支对话
minigames	23	2428	四个小游戏与共用 2D 引擎
authorize	8	1142	注册、登录、会话与权限
about	4	690	项目介绍与制作组页面
achievements	9	579	成就定义、判定与展示
合计	92	15677	

与第三章的清单一一对应。

4.5 质量保证

4.5.1 异常与降级

原则是所有可能失败的外部调用都返回结构化错误码，由调用方转成玩家可读的中文提示，不让异常中断流程。

- **本地存储不可用。**浏览器的隐私模式或配额写满都会让读写抛错，所有读写都包在 try/catch 里，失败统一返回 STORAGE_UNAVAILABLE，界面提示“浏览器存储暂时不可用”，游戏本身不崩溃。
- **音频自动播放被拦截。**audio.play() 的失败被捕获为 AUDIO_PLAY_BLOCKED，音乐改为等待玩家的第一次点击或按键后起播，界面上另有常驻的静音开关。
- **过场视频无法播放。**播放器的结束、出错、中断三个事件绑定同一个收尾函数，play() 的失败也走这条路径。任何失败都让剧情直接继续，玩家可能看不到某一段过场，但不会停在黑屏前。
- **存档版本不匹配。**读到版本号低于当前格式的存档时返回 SAVE_VERSION_UNSUPPORTED，提示玩家开始新游戏，旧数据保留在原处不删除。

有一处刻意不做降级：小游戏的渲染循环在检测不到 requestAnimationFrame 时直接抛错停止。项目假定运行环境是现代浏览器，这里做静默兜底只会把问题推迟到更难定位的地方。

4.5.2 测试

测试代码在 `test/` 目录下，35 个文件、6122 行，分两层。

逻辑层用 Node 内置的 `node:test` 运行单元测试，覆盖存档读写与版本兼容、账户与权限、主线与探索之间的接口契约、小游戏规则与成就判定。

页面层用 Playwright 驱动浏览器回归脚本，脚本自带本地静态服务器，真实打开游戏页面走一遍主流程，检查存档写入与页面状态。这一层覆盖单元测试看不到的部分：脚本加载顺序、模块之间的实际联动，以及存档在真实浏览器中的落盘结果。

另有一份主线引擎的冻结副本放在 `test/exploration/vendor/` 下，探索模块针这份副本做契约测试。主线改动接口时，契约测试会在合并前失败，避免上游的改动无声地破坏下游。

详细的测试结果见第六章。

4.5.3 协作流程与 subPR 分工

版本管理使用 GitHub Flow。`main` 是唯一长期分支，始终保持可运行、可演示的状态；功能在短期 Feature 分支上开发，一个分支对应一个 subPR，只解决一个需求，合并即交付。

每个 subPR 在合入前经过技术审查与需求验收两道检查。技术审查关注实现是否清晰、有没有引入不必要的依赖；需求验收对照该 subPR 对应的需求条目逐项确认。审查在 GitHub 上进行，讨论记录随 PR 保留，后续出现问题时可以回溯当时的判断依据。

详细的成员分工与进度安排见第五章。

五、项目管理与进度

5.1 项目提出与开发方式

9 月 2 日，五名成员各自提出选题方向，逐一陈述并互相质询，当天确定最终方案，并完成仓库初始化。项目定名为《借灯》，制作组名白灯客。

9 月 3 日确定开发方式：需求驱动的 subPR 闭环。整条链路固定为五步，提出需求、编写需求文档、需求评审确定边界与实施方式、开发、测试与集成。开发方式确定之后，四个版本的迭代都按这条链路推进，不再中途改动。

5.2 协作流程

版本管理采用 GitHub Flow。`main` 是唯一长期分支，始终保持可运行、可演示的状态；功能在短期 Feature 分支上开发，一个分支对应一个 subPR，只解决一个需求，合并即交付。

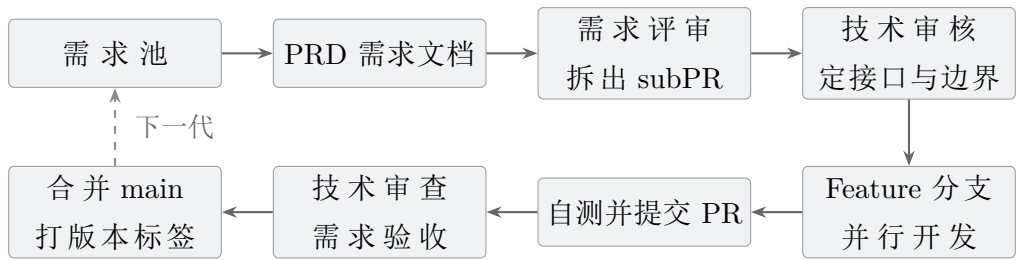


图 5.1 subPR 闭环流程

流程的每一步都有明确的输出物：需求池的条目进入 PRD 成为可评审的需求，需求评审把一个需求拆成若干 subPR 并确定边界，技术审核确定模块接口与分支策略，开发在各自的 Feature 分支上并行进行，合并前必须同时通过技术级测试与需求级验收。合并之后回到需求池，作为下一代需求的输入。

5.2.1 subPR 的运作规则

1. 每个 subPR 从最新 `main` 创建短期分支，命名为 `feature/` 加版本、需求 ID 与简述。
2. 一个分支只服务一个需求，提交记录说明实际变化。
3. 提交评审前先拉取最新 `main`，解决冲突并重新完成自测。

- 4. PR 描述关联需求 ID、验收条件、测试证据、界面截图与已知问题。
- 5. 合并前同时满足技术级测试与需求级验收。
- 6. 合并后删除已完成的 Feature 分支，全部 subPR 完成后为版本打标签。

不建立长期 dev 分支；不允许在个人分支上积压到最后一天再一次性合并。

5.2.2 五日迭代节奏

一个迭代周期为五个工作日，每天的产出固定，不随人员轮换改变。

表 5.1 迭代日程

迭代日	活动	时长	核心产出
第 1 天	需求评审日	全天	确定本周 PRD，含需求清单、subPR 边界与验收条件；完成故事细节确认与素材需求梳理
第 2 天	技术审核日	全天	技术负责人确认接口定义、模块边界与分支策略；各模块负责人完成设计方案并互审
第 3-4 天	开发日	两天	各 subPR 并行开发、自测；每日 09:00 站会同步进度；每日合并前完成 PR 审查与功能验收
第 5 天	讨论复盘日	全天	汇总本周成果，验收测试，整理遗留问题与风险；输出下一代 PRD 输入材料，完成角色轮换交接

需求评审日对需求池中的待开发条目做三项决策：确认优先级（P0 阻塞项、P1 核心功能、P2 优化项）、识别依赖关系（哪些需求阻塞其他需求、哪些可以并行）、把需求拆成 subPR 并逐一分配到负责人。开发日按 P0 优先、P1 随后、P2 弹性安排的顺序推进。

开发日的站会上，每人依次回答三个问题：昨天完成了什么，今天计划做什么，遇到了什么阻塞。前两项对照 subPR 进度核对，第三项由技术负责人和项目经理当场协调。站会之后，项目经理把当日的任务调整同步到 subPR 清单。

5.3 轮值角色与分工

项目采用轮值角色制。一轮中必须有六类责任，但角色不固定到人，按代轮换。

轮值规则有五条：

- 1. 以代为单位轮动。一代需求全部落地并打版本标签后交接，下一代由新的人员接任。

表 5.2 轮值角色权责

轮值角色	开发前	开发中	合并与交接
项目经理	汇总上一代结果；起草下一代 PRD；主持需求评审；确定 subPR 边界与依赖	跟踪需求状态；处理范围冲突；组织必要的需求变更	确认本代需求全部落地；整理版本记录、遗留问题和下一代输入
产品经理	检查用户流程、故事边界和验收标准；防止提前剧透或虚报完成	回答需求问题；检查实现是否偏离玩家体验	按 PRD 做需求级验收；通过后允许合并，不通过则写明差距
技术负责人	确定模块边界、接口和分支策略；识别技术依赖	审查关键实现；协调冲突；保证 main 可运行	拉取最新 main 做技术级测试；检查代码、兼容性、错误反馈和测试证据
模块负责人 / 开发者	领取一个可独立交付的 subPR；确认范围与依赖	在 Feature 分支实现、自测、记录问题；及时同步上游变化	修复评审问题；补齐 PR 说明；完成交接材料
内容与视觉责任人	核对大纲、剧透边界和素材需求；统一画面基调	编写或校对剧情数据，制作和压缩素材，检查文字可读性	核对成品内容与素材授权信息，不让占位素材进入发布版
测试记录人	根据验收条件准备场景、设备和异常数据	记录成功与失败路径，复测修复项	汇总回归结果，确保问题有负责人、有状态、有复现步骤

- 2. **完成即换代。**轮动不等待固定日期。本代提前完成就立即进入下一代需求评审，未通过验收则继续修正，不因周次变化宣布完成。
- 3. **模块在换代时轮动。**成员优先承担与上一代不同的功能域，变更后的负责人写入新一代 PRD 后才生效。
- 4. **作者不能独自验收自己。**subPR 的开发者不能同时是该 PR 唯一的技术审查人或需求验收人。
- 5. **轮换不切断责任。**前任交付 PRD、已合并 subPR、测试记录、遗留问题与最新 main 状态，继任者确认后才开始下一代。

分工按功能域拆分，再按依赖安排并行或串行。功能域分为七块：站点与账

户、剧情与探索、状态与存档、背包与成就、小游戏、内容与视觉、文档与测试。每个模块必须有明确负责人，不出现由全组负责却无人处理的情况。跨模块接口由相关负责人在技术审核日共同确认，例如剧情引擎向探索系统提供节点与调查状态，小游戏把完成结果交回统一状态与存档接口。

5.4 迭代进度

四个版本首尾相接，上一代收尾时下一代已经开始，因此相邻版本的时间区间有重叠。每代结束都产生一个可运行、可演示、可验收的增量。

表 5.3 版本时间轴

版本	时间	核心工作	交付成果
V1	9.3–9.8	需求评审、仓库搭建、目录规范化、页面骨架、基础交互、前置剧情、存档、首个地图小游戏	完整项目架构、13 个页面、可连续游玩的游玩链路、存档体系
V2	9.7–9.11	首轮岗位轮换、PRD 更新、subPR 拆解、中段剧情、新增小游戏、线索与成就扩展、存档兼容	中段完整剧情、新增交互功能、成就体系、历史存档兼容
V3	9.9–9.15	二次轮换、终局剧情、证据抉择系统、多结局落地、全量测试、文档完善、版本归档	全剧情分支、五个结局、完整成品项目、标准化文档
V4	9.16–9.20	剧情定稿删改、转场效果、过场视频接入、对话框与响应式、小游戏第二阶段、界面细节调整	定稿剧情文本、11 处转场、7 段过场视频、四个小游戏全部完成

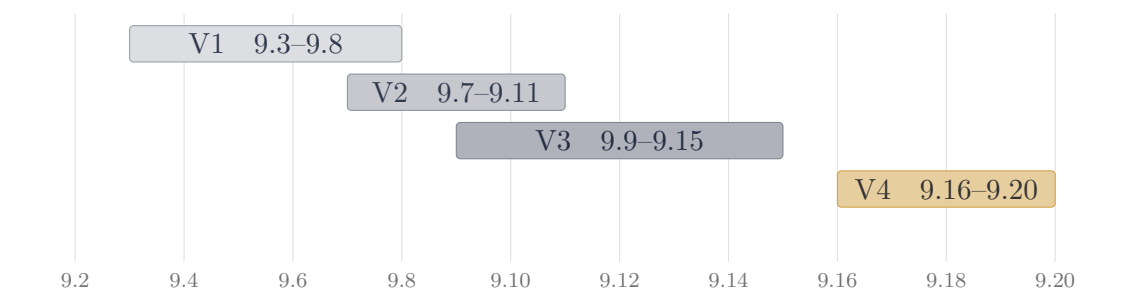


图 5.2 四个版本的迭代时间轴

5.5 流程效果

这套流程在四个版本中保持稳定，效果集中在四点上。

需求驱动。每个版本的起点是需求评审而不是任务分配，subPR 从评审通过的 PRD 里拆出来。需求在评审中完成增删改与合并，并且必须回写到当代 PRD 与 subPR 清单，不留在聊天记录里。复盘日的总结又是下一代的输入，四个版本因此是连着的，不是四段各自为政的工作。

边界清晰。每个 subPR 只解决一个需求，范围在开发前就确认。模块之间通过接口约定通信，改动其中一块不需要通知其他模块。

分工明确。六类责任在每一代都落到具体的人，并且记录在当代 PRD 与 PR 中。轮换之后仍可以查到某一代是谁做的技术审查、谁做的需求验收。

并行友好。各 subPR 在独立分支上并行开发，冲突集中在合入时刻处理，日常开发时间没有被合并工作占用。开发日的每日合并进一步把冲突摊平到每一天，避免积压。

从 9 月 2 日立项到 9 月 20 日，主线累计 229 次提交，共开出 73 个合并请求，其中 68 个已合入。这四个版本的划分与逐日推进的节奏，是这套数据能够形成的原因。

六、测试与回帖情况总结

6.1 浏览器兼容测试



图 6.1 chrome 浏览器



图 6.2 edge 浏览器

6.2 操作系统兼容测试

6.3 分辨率兼容测试

【dddddddddddddddddd
dddddddddddddddddd
dddddddddddddddddd
dddddddddddddddddd
dddddddddddddddddd

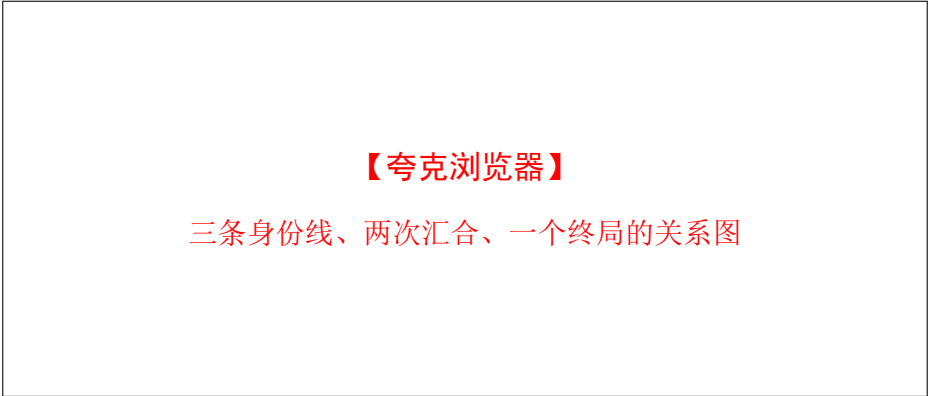


图 6.3 夸克浏览器

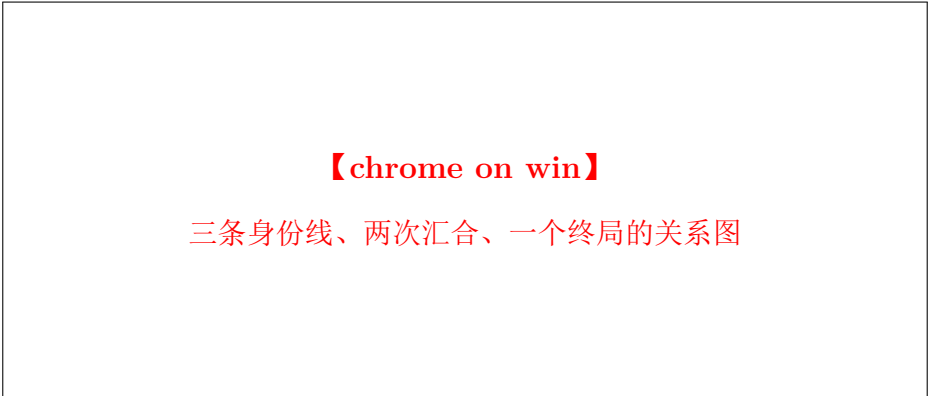


图 6.4 Windows

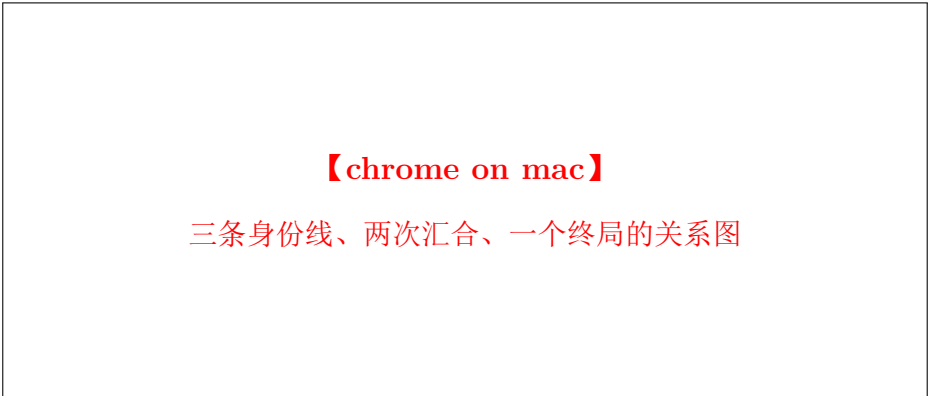


图 6.5 Mac

dddddddddddddddddddddd
dddddddddddddddddddddd
dddddddddddddd**】**

6.4 回帖情况汇总

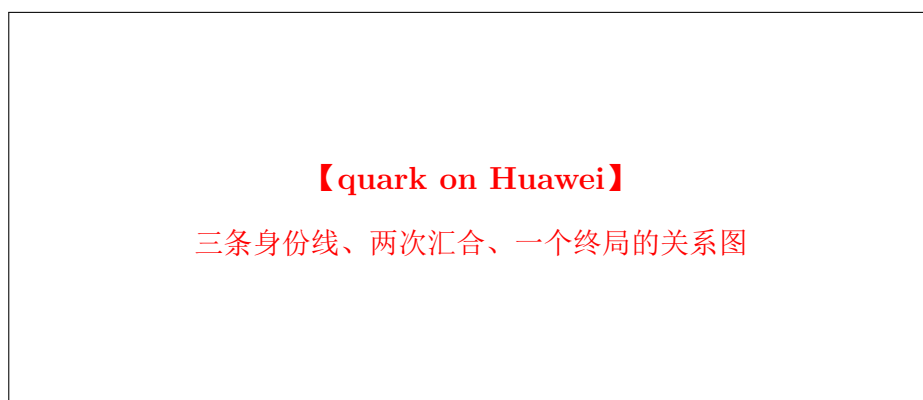


图 6.6 Android

4班-白灯客组-白灯客 杨梦 - 2026年09月5日 星期六 14:11 《白灯客》是一款以文化者调查古村“借灯”传说为开端的网页交互解谜游戏。玩家在阅读、探索和选择中逐步发现自己就是那个制造恐怖的人。游戏以多身份叙事和层层递进的反转，探讨责任追溯与自我救赎的人性主题。	
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月5日 星期六 14:12 4班-白灯客组-项目计划 📎 4班-白灯客组-项目计划.pdf	
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月5日 星期六 14:14 4班-白灯客组-邀请宋河网站 📎 4班-白灯客组-邀请宋河.rar	
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月5日 星期六 14:31 4班-白灯客组-项目日志与会议记录 📎 4班-白灯客组-项目日志与会议记录.pdf	
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月6日 星期日 01:23 小组介绍视频 📎 4班-白灯客组-小组介绍视频.mp4	
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月13日 星期日 00:41 4班-白灯客组-项目计划与设计规范 📎 4班-白灯客组-项目计划.pdf	回复：4班-白灯客组-白灯客 杨梦 - 2026年09月6日 星期日 01:23 小组介绍视频 📎 4班-白灯客组-小组介绍视频.mp4
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月13日 星期日 11:25 4班-白灯客组-个人日志套装 📎 4班-白灯客组-个人日志（第二周）.pdf	回复：4班-白灯客组-白灯客 杨梦 - 2026年09月6日 星期日 14:31 git repo 链接: https://github.com/214notfound/jiedeng 📎 4班-白灯客组-游戏原型V1.rar
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月13日 星期日 11:27 4班-白灯客组-游戏原型与测试视频 📎 4班-白灯客组-游戏原型与测试视频.rar	回复：4班-白灯客组-白灯客 于知让 - 2026年09月9日 星期三 08:11 今天周三进行集中开发
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月14日 星期一 13:11 git repo链接: https://github.com/214notfound/jiedeng	回复：4班-白灯客组-白灯客 杨梦 - 2026年09月11日 星期五 17:05 电影欣赏网站 📎 4班-白灯客组-电影欣赏网站.rar
回复：4班-白灯客组-白灯客 杨梦 - 2026年09月15日 星期二 20:53 第三周汇报计划与PPT 📎 4班-白灯客组-汇报计划.docx 📎 4班-白灯客组-项目汇报ppt.pptx	回复：4班-白灯客组-白灯客 杨梦 - 2026年09月11日 星期五 17:07 4班-白灯客组-中华美食网站 📎 4班-白灯客组-中华美食网站.rar
回复：4班-白灯客组-白灯客 于知让 - 2026年09月17日 星期四 15:32 9.16删情v4的pr结束	回复：4班-白灯客组-白灯客 杨梦 - 2026年09月13日 星期日 00:40 4班-白灯客组-第二周项目日志与会议纪要 📎 4班-白灯客组-第二周项目日志与会议纪要.pdf

图 6.7 回帖情况

七、最终显示效果

【待补充】

八、小组总结

9.1 项目开发全景复盘：从 0 到 1 的协作落地在“程序设计与制作”课程大作业的驱动下，我们“六个基米哈一天”小组以《第五人格》为 IP 原型，耗时三周完成网页小游戏《第五人格：推演终幕》的开发。项目以“求生者剧情推演 + 监管者卡牌战斗”为核心玩法，涵盖剧情系统、战斗系统、存档系统、地图交互等六大核心模块，最终交付包含 3 万字剧情、17 个人物档案、2 张可切换地图、1 个修机小游戏及豆瓣电影 / 中华美食两个附属网站的完整成果。回顾开发全流程，我们以“每日会议为协作枢纽、分工互补为推进引擎、问题共解为突破关键”，实现了从需求拆解到功能落地的高效推进。

9.1.1 第一周：框架搭建与基础攻坚（8.24-8.30）

项目启动阶段，我们聚焦“定方向、搭框架、破初障”三大目标。8 月 24 日 - 25 日，组长赵硕涵牵头完成组队与方向决策，组织成员学习项目 PPT 后，经线下会议投票确定开发“《第五人格》IP 二创交互游戏”，明确“剧情 + 战斗”双核心玩法；房金衡作为剧情与战斗策划，整合成员创意形成第一版游戏大纲，规划“序章 + 四章主线”剧情结构，定义“求生者推演（剧情向）+ 监管者战斗（卡牌向）”的核心循环；梁思齐、刘雨桐、潘治鑫组成技术组，同步启动前端基础学习，王宇豪则单美术线推进，搭建像素素材库。8 月 26 日 - 30 日，各模块进入基础开发阶段。技术组方面，梁思齐从前端三件套入门，克服 CSS 语法误区（误写 `tiles` 类后改用 `button` 标签），完成登录界面与角色移动类初步开发；刘雨桐聚焦成就系统，参考往届“肖申克”“瑞雪”等作品，拆解出“`Achievements.json` 配置表 + `AchievementManager` 管理逻辑 + 弹窗 UI”的模块架构，同步编写剧情对话界面；潘治鑫则钻研 `localStorage` 存储逻辑，为存档系统奠定基础。剧情与战斗组方面，房金衡完成序章“初入庄园”剧本及第一章前两幕剧情，同时设计战斗系统基础规则，定义 90 点初始生命值、2 点行动点的玩家参数，规划普通攻击、利剑之刺等 12 类基础卡牌。美术组方面，王宇豪从零学习 `Tiled Map Editor` 与 `Aseprite`，完成首个测试地图与主角像素小人初稿，解决素材风格不统一问题。此阶段，我们遭遇首个技术难关——CORS 协议跨域问题。8 月 29 日，梁思齐、潘治鑫在调试地图切换与存档加载时，发现直接打开 HTML 文件无法获取数据，经每日会议集体排查，最终参考往届方案，通过 Python HTTP 服务器与 Live Server 插件双路径解决，同时总结出“避免绝对路径、统一相对路径调用”的开发规范，为后续模块协作扫清障碍。

9.1.2 第二周：功能深化与协同突破（8.31-9.6）

进入核心开发阶段，我们以“模块协同、问题共解”为核心，推动项目从“能用”向“好用”升级。技术组实现多系统联动：梁思齐优化角色移动逻辑，修复“对话未完成关闭后状态异

常”“空气墙不精准”等 5 类 Bug，同时创新性地用 Web Audio API 将地图钢琴改造为可交互小游戏，解决“Click Noise”杂音问题；刘雨桐重构剧情系统架构，将 20 个场景剧情拆分至独立 JS 文件，通过 scriptEngine.js 统一接口，解决浏览器加载限制（加入 datamanager 与 UCL 参数加载函数）；潘治鑫则分离成就与存档系统 42 项目计划统（避免前期耦合导致的维护难题），新增自动存档功能，实现“剧情跳转后角色位置与背包状态无损恢复”。剧情与战斗组实现内容扩容：房金衡完成第一章收尾（厂长剧情）与第二章审核，同步优化战斗系统，为监管者新增特色机制（如红夫人“重生”、蜘蛛“蛛网反伤”），调整卡牌数值避免流派失衡；同时拓展附属项目，耗时 10 余小时用 Python（requests+lxml）爬取豆瓣电影 Top250 数据，突破反爬限制（设置 1 秒延迟、伪造 headers），将电影名称、评分、海报等信息以 JSON 格式存储，完成电影欣赏网站开发。美术组则推进地图迭代：王宇豪完成地下室地图第二版，优化房间布局（划分图书区、药水区），通过 GIMP 缩放处理 16×16 与 32×32 素材比例问题，同时绘制带感叹号的交互贴图，提升场景引导性。此阶段的关键突破在于“跨模块协作机制”的建立。9 月 2 日，梁思齐在开发钢琴小游戏时，发现 iframe 与角色移动的鼠标聚焦冲突，经与王宇豪（美工）、潘治鑫（存档）召开专项会议，最终实现“点击游戏区域控制小游戏、点击下方控制角色”的分层交互逻辑；9 月 4 日，刘雨桐因数学建模竞赛请假前，提前完成剧情系统文档梳理，确保潘治鑫能顺利对接存档接口，体现“无缝交接”的协作意识。

9.1.3 第三周：整合优化与成果交付（9.7-9.13）收尾阶段，我们聚焦“全流程测试、细节优化、成果整合”，确保项目符合课程交付标准。技术组完成系统闭环：梁思齐解决战斗系统与存档的数据断联问题（通过日志追踪定位参数传递时机错误），实现“战斗胜利后自动触发剧情、失败后重置状态”的完整流程；潘治鑫开发修机小游戏，设计“校准进度条 + 钥匙解锁门”的地图衔接逻辑，同时完成成就系统与结局的联动（Boss 战胜利触发“欧律狄刻”成就，失败触发“故事的最后”成就）；刘雨桐则完成角色档案系统开发，通过 localStorage 实现“账号级解锁状态存储”，同步导入全部 3 万字剧情，解决文件命名格式错误（如“1.jpg”误判为二进制文件）。剧情与美术组完成内容收尾：房金衡撰写第四章双结局（“精神清醒”“永坠混沌”），补充 17 个人物档案（含生日、特质、背景故事），同时调整战斗数值（如作曲家“音韵”被动技能改为“回合开始抽 1 张牌”）；王宇豪则对地下室地图进行像素级优化，为关键道具（实验笔记）添加 Aseprite 发光动画，校准所有碰撞框，确保场景交互无死角。最终交付阶段，我们建立“模块测试 - 交叉验收 - 整体联调”的三级质检体系：赵硕涵统筹全流程测试，重点验证地图切换、存档加载、成就触发等关键路径；各成员交叉测试非负责模块（如梁思齐测试剧情跳转、王宇豪测试战斗 UI），最终解决“非 Live Server 环境钥匙获取异常”“结局页面音效不同步”等 8 类遗留问题，确保项目稳定运行。

9.2 技术难关攻克：从“卡壳”到“破局”

的能力跃迁三周开发中，我们遭遇的技术挑战不仅是“问题清单”，更是团队技术认知与协作能力升级的“催化剂”。每一次难关的突破，都伴随开发规范的完善与技术思维的深化。

9.2.1 跨域与资源加载：从“被动规避”到“主动规范”

CORS 协议与浏览器资源加载限制是项目初期最棘手的问题。8 月 29 日，梁思齐在调试地图矩阵（用数字标记交互区域）时，发现直接打开 HTML 文件无法读取 JS 配 43 项目计划置，控制台报错“Access to script at 'file:///...' from origin 'null' has been blocked by CORS policy”；同期，潘治鑫的存档系统也出现“localStorage 数据无法跨页面共享”的问题。初期，我们尝试修改文件路径、改用绝对路径调用，均以失败告终。在 8 月 29 日晚间会议中，我们转变思路：先调研往届解决方案（发现“瑞雪”项目用 Python HTTP 服务器启动），再分工测试不同方案——潘治鑫尝试用 Python 的 http.server 模块（命令“python -m http.server 8000”），成功解决跨域问题；梁思齐则测试 Live Server 插件，发现能自动处理资源加载。但新问题随之而来：不同成员使用不同启动方式，导致代码路径调用混乱。为彻底解决此问题，我们共同制定“资源调用三规范”：一是所有文件统一使用相对路径；二是建立“assets”目录分类管理 JS、CSS、图片资源；三是在 readme 文档中明确“Python HTTP 服务器与 Live Server 双启动方式”的操作步骤。此规范不仅解决了当前问题，更避免了后续多模块协作时的路径冲突，体现“问题解决 - 规范沉淀”的技术思维升级。

9.2.2 多系统联动：从“各自为战”到“接口协同”

随着模块增多，“系统间数据同步”成为新挑战。9 月 11 日，梁思齐在对接战斗系统与地图主系统时，发现战斗结果无法回传至主游戏，导致玩家完成战斗后卡在结算界面。经排查，问题根源在于：战斗系统的结果参数仅在本地存储，未通过存档接口同步至主游戏的状态；同时，主游戏未监听战斗页面的关闭事件，无法主动拉取最新数据。针对此问题，我们成立专项攻坚小组（梁思齐、潘治鑫、房金衡），通过三次短时会议确定解决方案：一是在战斗系统的“战斗结束”函数中，新增调用存档接口的逻辑；二是主游戏页面添加“onmessage”监听，接收战斗页面的关闭通知后，拉取最新数据；三是定义统一的 URL 参数格式，确保参数传递无歧义。此次攻关让我们深刻认识到“接口设计先行”的重要性。此后，我们在新增模块前，都会先在每日会议中确定接口参数，避免后期返工。这种“先定义接口、再开发功能”的思路，使后续修机小游戏与地图门解锁的对接仅用 2 小时完成，效率提升 609.2.3

像素美术与技术适配：从“风格割裂”到“视觉统一”

美术与技术的适配问题贯穿开发全程。王宇豪作为美工，初期面临“素材风格不统一”“像素比例失调”两大难题。为解决这些问题，我们建立“美术 - 技术协同机制”：一是素材标准统一，王宇豪明确“所有素材采用 32×32 像素、统一暗色调”，并用 GIMP 批量处理；二是交互区域可视化，梁思齐在 Canvas 绘制时，为可交互物件添加半透明红色边框（测试模式下显示），方便王宇豪校准美术位置；三是效果迭代反馈，王宇豪为实

验笔记添加发光动画后，梁思齐同步调整触发范围，确保玩家能清晰感知交互点。最终，地下室地图的视觉与技术适配效果显著：场景风格统一，交互准确率达 100 次协作也让我们明白：美术不仅是“视觉装饰”，更是“交互引导”——王宇豪后期为关键道具添加的发光动画，使玩家找到实验笔记的平均时间从 2 分钟缩短至 30 秒，极大提升游戏体验。

44 项目计划 9.3 协作认知升级：从“分工”到“共生”

的团队蜕变三周开发不仅是项目落地的过程，更是团队协作认知从“简单分工”向“深度共生”跃迁的过程。我们深刻体会到：优秀的团队不是“各做各的”，而是“彼此支撑、互为补充”。

9.3.1 每日会议：从“进度同步”到“问题共治”

我们坚持的“每日 19 点线上会议”，是团队协作的“核心枢纽”。会议初期以“各自说进度”为主，效率较低；后改革为“问题优先”模式——每次会议先聚焦 3 个核心问题，再同步进度、分配任务。这一改革带来显著变化：针对 CORS 跨域问题，会议仅用 40 分钟就确定解决方案；讨论战斗难度梯度时，技术、策划、美术共同形成“章节 - 角色 - 数值 - 视觉”四维联动的难度设计。改革后的会议解决问题效率提升 150 要的是，每日会议培养了“责任共担”的意识。这种“补位不缺位”的协作，让项目在成员有临时任务时仍能高效推进。

9.3.2 分工与补位：从“固定角色”到“动态支撑”

项目初期，我们按“技术、策划、美术”划分固定角色，但发现“绝对分工”无法应对复杂需求。为此，我们转向“核心角色 + 动态补位”的模式：每个成员有明确核心职责，但同时需掌握其他领域的基础技能，以便在团队需要时补位。这种模式成效显著：潘治鑫（核心存档）协助优化移动逻辑，用“状态机”思路重构代码；王宇豪（核心美术）学习基础 HTML/CSS，协助优化页面排版，使响应式适配率从 70 态补位”不仅提升了项目效率，更让每个成员看到“自身工作的价值关联”，形成良性协作循环。

9.3.3 冲突与共识：从“意见分歧”到“最优解共建”

开发中并非毫无冲突，而是通过“理性讨论、数据支撑”将冲突转化为“共识升级”的契机。关于“战斗系统是否需要难度梯度”的分歧，我们采取“数据验证 + 最小原型”的方案：房金衡计算战斗回合数，梁思齐快速制作“配置表原型”，最终团队达成共识——采用“配置表 + 动态加载”方案，既实现难度梯度，又避免代码冗余。此次冲突让我们建立“冲突解决三原则”：一是不回避分歧；二是用数据说话；三是做最小原型。这一原则确保团队决策始终基于“可行性”与“玩家体验”，而非个人偏好。

9.4 项目开发认知：从“功能堆砌”到“用户体验”的思维转变

三周开发让我们对“游戏开发”的认知实现从“技术驱动”到“用户为中心”的深刻转变。我们逐渐明白：优秀的游戏不仅是“功能的集合”，更是“体验的闭环”。

9.4.1 从“实现功能”到“优化体验”：细节决定成败

初期，我们更关注“功能是否实现”，而忽略“体验是否流畅”。例如，角色移动在高刷新率电脑上过快，成就弹窗会遮挡战斗界面。针对测试反馈，我们启动“体验优化专项”：梁思齐用“requestAnimationFrame”替代固定时间间隔，并添加移动加速度，使移动手感更

佳；刘雨桐优化成就弹窗，新增“战斗中延迟显示”逻辑，并调整位置至右 45 项目计划下角。体验优化后的效果显著：第二次测试中，玩家对“移动流畅度”的满意度从 40 提升至 909.4.2 从“开发者视角”到“玩家视角”：换位思考的重要性开发初期，我们常陷入“开发者视角”的误区，默认玩家“懂我们的设计逻辑”。例如，地图初期无任何引导，导致 70 家视角验证机制”：一是引入外部测试者；二是推行“自我盲测”；三是简化信息传递（添加“剧情回顾”按钮和“光效引导”）。“玩家视角”的转变带来显著效果：外部测试者找到实验室的平均时间从 15 分钟缩短至 3 分钟。这一过程让我们明白：游戏开发不是“自我表达”，而是“与玩家的沟通”。9.4.3 从“单打独斗”到“工程化思维”：项目管理的价值项目初期，我们缺乏工程化思维，导致代码无规范、文档无记录、版本无管理。为解决这些问题，我们逐步建立“工程化开发体系”：一是制定《前端开发规范》，统一代码风格；二是推行文档管理，每个模块完成后编写技术文档；三是采用“日期 + 版本号”的命名规则进行版本控制。工程化体系的建立，使项目后期维护效率大幅提升。这让我们认识到：小型项目同样需要工程化思维——规范不是“束缚”，而是“效率保障”；文档不是“多余工作”，而是“团队协作的桥梁”。9.5 总结与展望：不止于“完成作业”，更是“能力成长”三周《第五人格：推演终幕》的开发，对我们而言不仅是完成课程作业，更是一次完整的“小型项目实战”。我们收获的不仅是一个可运行的游戏作品，更是技术能力、协作意识、项目思维的全方位成长。回顾整个开发过程，我们也清醒地认识到不足：

- 原创美术能力不足：王宇豪虽完成基础素材，但原创角色立绘仍有提升空间。
- 游戏可玩性深度不够：战斗系统虽实现基础卡牌玩法，但缺乏“卡组流派”“角色组合策略”等深度设计。
- 工程化体系仍需完善：未使用 Git 进行版本控制，多成员协作时仍有冲突风险。

未来，若有机会迭代项目，我们计划从三方面改进：

- 提升美术原创性：学习 Aseprite 高级绘画技巧，为每个角色设计独特的战斗动作。
- 深化玩法设计：新增“卡组预设”“角色羁绊”系统，提升战斗策略性。
- 完善工程化工具：引入 Git 进行版本控制，使用 Figma 进行 UI 设计协作，进一步提升团队开发效率。

此次项目开发，让我们深刻体会到：“做项目”是最好的学习方式。这三周的经历，将成为我们后续学习与工作中宝贵的财富，激励我们继续在“技术与协作”的道路上不断探索、不断成长。

九、个人总结

【待补充】

十、评优申请

【待补充】

十一、最终得分

【待补充】